

Adaptive Evidence-Weighted Early-Exit Naive Bayes: Reducing Inference Cost via Feature Ordering, Discriminative Thresholding, and Confidence-Based Stopping

Tollamadugu Siva Naga Venkata Raviteja

August 2026

Published on Zenodo — DOI: 10.13140/RG.2.2.23508.97921

Code: github.com/RAVITEJA12158/NaiveBayes_Research

License: CC BY 4.0

Version 2 (revised) — see Section 7 for what changed since the v1 DOI above

Abstract

This paper reports a mixed result: an inference-time speedup technique for Naive Bayes that works well on small datasets and is matched or beaten by a much simpler baseline on larger ones. We treat that outcome, and the process of discovering it, as the contribution. Naive Bayes classifiers evaluate every feature for every class at inference time, even though many features carry little discriminative information for a given input. We propose Adaptive Evidence-Weighted Early-Exit Naive Bayes (AEWEE-NB), an inference-time modification of Gaussian Naive Bayes that (i) orders features by a training-time class-separability score, (ii) skips individual features whose per-sample discriminative weight falls below a threshold τ , and (iii) halts evaluation once the gap between the leading and runner-up class log-scores exceeds a confidence margin δ . Component (iii) is a direct application of Wald's Sequential Probability Ratio Test (1945) to a Naive Bayes score accumulator; we make no claim to novelty there and state this plainly rather than presenting early-exit as new. We implement the method and two baselines — standard Gaussian Naive Bayes, and a deliberately simple static top-k feature-selection classifier with no per-instance adaptivity at all — from the same numerical primitives, and evaluate all three on four small-to-moderate benchmark datasets (13–100 features, Wine / Breast Cancer / Digits / a synthetic set) plus three larger synthetic datasets (200–2,000 features, up to 50,000 samples) designed to probe concentrated vs. diffuse discriminative signal. At small scale, AEWEE-NB reduces the average number of per-sample feature evaluations by $1.9\times$ to $5.4\times$ relative to standard Naive Bayes with no accuracy loss (and a 6-percentage-point accuracy gain on one dataset). At larger scale, once the static top-k baseline is included, the picture changes: static top-k matches or exceeds AEWEE-NB's accuracy at every matched compute budget we tested, and is clearly better in the high-dimensional, sparse-signal regime that most resembles real Naive Bayes deployments (86.8% accuracy using 5% of features vs. AEWEE-NB's 84.2% using 17.4% of features). An ablation study further shows that ordering and thresholding, on their own, save no computation at all — early-exit does essentially all of the work, with ordering and thresholding serving only to make it safe. This is a revised version of the paper: Section 7 reports two bugs subsequently found in our own hyperparameter-selection process (a safety check that rewarded noisy candidates, and an unsafe fallback), and shows that correcting both changes the static-top-k comparison from a uniform loss to a genuine mixed result — AEWEE-NB wins on two of four datasets once selection is done properly. We report all of this, including the results that do not favor the proposed method and the bugs found in our own evaluation code, as the paper's main contribution: a rigorous, self-correcting accounting of when a per-instance adaptive-computation scheme for Naive Bayes earns its complexity over the simplest fixed alternative, and when it does not.

Keywords

Naive Bayes, early-exit inference, anytime classification, feature selection, cost-sensitive classification, adaptive computation

1. Introduction

Naive Bayes classifiers remain widely used where inference latency, memory, or energy budgets are tight — embedded and edge devices, high-throughput filtering pipelines, and large-scale streaming systems — because they are simple, well-calibrated under their independence assumption, and cheap relative to most alternatives. "Cheap" is relative, however: standard Naive Bayes still evaluates every feature for every class on every input, even when a small subset of features would already determine the prediction with high confidence. This is analogous to the observation that motivates early-exit neural networks (Teerapittayanon et al., 2016) and anytime ensemble prediction: most inputs do not need the full computation budget to be classified correctly, and only a minority of "hard" examples do.

This paper develops and empirically evaluates a concept for adapting that idea to Naive Bayes: order features by how discriminative they are, skip features that turn out to be uninformative for the specific input being classified, and stop as soon as the accumulated evidence is decisive. We call this Adaptive Evidence-Weighted Early-Exit Naive Bayes (AEWEE-NB). The three mechanisms are individually simple, but their interaction is not obvious a priori — for instance, does thresholding actually save computation, or merely change which features contribute to the final score while the cost of deciding is still paid? We treat this as an empirical question rather than assuming the answer, and the ablation results in Section 5.5 show the answer is more subtle than the original concept assumed.

Contributions:

- A generalization of the evidence-weighting and early-exit rules from the binary case to arbitrary numbers of classes, using a variance-based discriminative-weight measure and a leading-vs-runner-up confidence margin.
- A from-scratch, numerically matched implementation of both the standard Gaussian NB baseline and AEWEE-NB, so that feature-evaluation counts are directly comparable (an early version of our own harness under-counted the baseline by a factor of C , the number of classes — we report this as a cautionary note in Section 4.4 since it is an easy mistake to reproduce).
- An empirical evaluation on four datasets with repeated-split robustness checks, an ablation study isolating each mechanism, and an honest accounting of when the method helps, when it does not, and why.

2. Related Work

Early-exit and anytime inference. The idea of stopping computation once a model is confident enough is well established for deep networks, where intermediate classifiers attached to early layers allow "easy" inputs to exit before reaching the final layer (Teerapittayanon, Bhattacharyya, and Chen, 2016; and subsequent work on dynamically configurable exiting). This literature consistently reports the same qualitative trade-off we observe: lower confidence thresholds save more computation but risk larger accuracy drops, and the optimal threshold is dataset-dependent rather than universal. Our early-exit rule is a direct transplant of this idea to a Naive Bayes score accumulator instead of a layered network.

Sequential hypothesis testing. More precisely, our early-exit rule is a special case of Wald's Sequential Probability Ratio Test (SPRT) (Wald, 1945), the classical statistical procedure that accumulates a log-likelihood ratio across sequentially observed samples and stops as soon as it crosses a decision threshold, and which is provably optimal in expected sample count for a given error rate (Wald & Wolfowitz, 1948). In the binary-class case, our stopping rule (accumulate a per-feature log-likelihood ratio across features, stop once the gap crosses δ) is structurally identical to SPRT, with 'samples over time' replaced by 'features in a fixed order.' We see the contribution here as narrower than 'inventing early-exit for Naive Bayes': it is applying a well-established sequential-testing principle to a setting (independent per-feature evidence rather than i.i.d. samples over time) where the ordering of evidence is itself a design choice, and empirically characterizing when that combination pays off. We think papers proposing early-exit or confidence-threshold stopping rules for classifiers should generally cite Wald (1945); the broader early-exit neural network literature above does not always do so, and we do not want to repeat that omission.

Cost-sensitive and budgeted feature acquisition. Lizotte, Madani, and Greiner (2003) study budgeted learning of Naive Bayes classifiers, where feature values are costly to acquire and must be purchased sequentially during training under a fixed budget — a training-time budget problem rather than the inference-time problem addressed here, but sharing the

core idea that not all features are worth their acquisition cost. Related sequential feature-acquisition formulations cast the problem as a POMDP or hidden Markov model over which feature to measure next, generalizing beyond the Naive Bayes independence assumption at the cost of a heavier formalism. Our method stays within the Naive Bayes assumption and treats feature order as fixed and learned once at training time, rather than solved per-instance, which keeps it cheap enough to be worth using in the first place.

Feature selection for Naive Bayes. Class-specific feature subset selection (e.g., the Explainable Class-Specific Naive Bayes classifier of Aguilar-Ruiz, Romero, and Cicconardi, 2024) reduces dimensionality per class ahead of training to improve interpretability and accuracy on high-dimensional genomic data. This is a static, training-time reduction; our threshold mechanism is dynamic and per-instance, deciding at inference time whether a given feature is worth including for the specific input rather than removing it globally.

Positioning. To our knowledge, the specific combination evaluated here — a global training-time ordering, a per-instance runtime threshold on a variance-based discriminative weight, and an early-exit rule based on the leading-vs-runner-up log-score gap, applied to multiclass Gaussian Naive Bayes — has not been reported as such in the literature we reviewed. The individual ingredients (ordering, thresholding, early stopping) are well preceded in adjacent settings, and we make no claim of inventing early-exit inference in general.

3. Method

3.1 Standard Gaussian Naive Bayes

Given a training set with C classes and n real-valued features, standard Gaussian Naive Bayes estimates a per-class mean $\mu_{\{k,i\}}$ and variance $\sigma^2_{\{k,i\}}$ for each feature i and class k , plus a class prior $P(C_k)$. For a test instance x , the predicted class is

$$\operatorname{argmax}_k [\log P(C_k) + \sum_{i=1}^n \log N(x_i; \mu_{\{k,i\}}, \sigma^2_{\{k,i\}})]$$

which requires computing $n \times C$ class-conditional log-densities per prediction. We treat one "feature evaluation" as the unit of cost: computing a feature's class-conditional densities for all C classes at once (a single vectorized operation in our implementation, and the natural unit since a Naive Bayes system typically holds all C per-class parameters for a feature together).

3.2 Global Feature Ordering

At training time we compute, for each feature i , a class-separability score using a Fisher-score-style statistic:

$$F_i = \operatorname{Var}_k(\mu_{\{k,i\}}) / \operatorname{Mean}_k(\sigma^2_{\{k,i\}})$$

i.e., the variance of the per-class means divided by the average per-class variance for that feature. Features with large between-class mean separation relative to their within-class spread receive a high score. Features are evaluated in descending order of F_i at inference time. This is a direct multiclass generalization of the original concept's proposal to rank features by an average absolute log-likelihood ratio (which is only well-defined for two classes).

3.3 Per-Sample Discriminative Weight and Thresholding

The binary formulation in the original concept weighs a feature, for a specific test instance, by the absolute log-likelihood ratio between the two classes. We generalize this to C classes as the variance, across classes, of the feature's log-density for the observed value:

$$w_i(x) = \operatorname{Var}_k(\log N(x_i; \mu_{\{k,i\}}, \sigma^2_{\{k,i\}}))$$

A feature whose log-density is nearly identical across all classes for this particular value of x_i is not helping discriminate the current instance, regardless of how discriminative it is on average, and is skipped if $w_i(x) < \tau$. A skipped feature simply contributes nothing to the score accumulator (equivalent to treating it as equally likely under every class).

Note on cost: computing $w_i(x)$ requires the same class-conditional density values that would be used to update the score, so evaluating whether to skip a feature costs the same as scoring it. Thresholding therefore changes which features are added to the accumulator, but by itself does not reduce the number of density computations performed — see Section 5.5 for the empirical confirmation of this point and its implication for where the real savings come from.

3.4 Early-Exit Stopping Rule

After each retained feature is folded into the running per-class log-score, we sort the scores and compute the gap between the top two classes. We stop and return the leading class once the gap exceeds a margin δ (log-score units), optionally also stopping once the softmax posterior of the leading class exceeds a target confidence. In the binary case this collapses to the log-likelihood-ratio stopping rule described in the original concept; the generalization is to track the runner-up rather than a single fixed comparison class.

3.5 Ablation Variants

To isolate each mechanism's contribution we implement four configurations, evaluated identically:

- Ordering only: features are visited in Fisher-score order, but none are skipped and no early stopping occurs (all n features are folded in, in a different order than the natural/standard order).
- Threshold only: features are visited in natural (unordered) order; features below τ are skipped; no early stopping.
- Early-exit only: features are visited in natural order, none are skipped, but evaluation halts once the score gap exceeds δ .
- Full method: ordering + thresholding + early-exit combined, as described above.

4. Experimental Setup

4.1 Datasets

We evaluate on four datasets chosen to span a range of feature counts, class counts, and feature redundancy structure:

Dataset	Features	Classes	n	Notes
Wine (UCI/sklearn)	13	3	178	Real, low-dimensional, near-linearly-separable
Breast Cancer Wisconsin (sklearn)	30	2	569	Real, moderate dimension, binary
Digits (sklearn, 8×8 handwritten digits)	64	10	1,797	Real, higher dimension, 10-way, correlated pixel features
Synthetic (make_classification)	100	4	4,000	25 informative + 25 redundant + 50 noise features, class_sep=1.2

Table 1. Datasets used in the evaluation. The synthetic dataset was substituted for Forest Covertype, whose download endpoint was unreachable from the sandboxed execution environment used for this study; make_classification lets us control redundancy and noise explicitly, which is arguably more informative for an ablation study.

4.2 Implementation

Both the standard baseline and AEWEE-NB are implemented from the same NumPy primitives (not wrapping scikit-learn's GaussianNB internally) so that the feature-evaluation accounting is identical and auditable. All features are standardized (zero mean, unit variance) before fitting, using statistics from the training split only. We deliberately did not use scikit-learn's built-in GaussianNB as the baseline implementation, to avoid any possibility of implementation-level differences (e.g., internal vectorization tricks) confounding the feature-evaluation comparison; we did, however, confirm our standard-NB baseline's accuracy is consistent with scikit-learn's GaussianNB on the same splits (within random-seed noise), as a sanity check.

4.3 Evaluation Protocol

For the main ablation grid, each dataset uses a single stratified 70/30 train/test split (fixed seed) and a grid over threshold $\tau \in \{0, 0.05, 0.2, 0.5\}$ and margin $\delta \in \{2, 5, 10\}$, for each of the four ablation variants in Section 3.5. Because a single split can be noisy, we separately re-ran the full method at one selected (τ, δ) operating point per dataset — chosen as the grid point achieving the best speedup subject to accuracy loss ≤ 0.5 percentage points on the initial split — across 10 independent stratified random splits (seeds 0–9), and report mean $\pm$ standard deviation for both accuracy and feature-evaluation count. This is not a fully nested cross-validation (the operating point is selected once, not re-selected per fold), so the robustness numbers should be read as a check that the main result is not a single-split artifact, not as an unbiased generalization estimate.

4.4 A Note on Fair Accounting

During development, our first version of this harness counted the standard baseline's cost as $n \times C$ (features $\times$ classes) while counting AEWEE-NB's cost as n (features only), which manufactured an artificial C -fold "speedup" that appeared even in ablation variants (ordering-only, threshold-only) that provably skip nothing and are mathematically identical in output to the baseline. We flag this because it is an easy and tempting mistake: any adaptive-computation paper reporting large speedups on multiclass problems should be checked for exactly this kind of unit mismatch. All results reported below use the corrected, apples-to-apples accounting (both models count one evaluation per feature, each of which costs C density computations in both algorithms).

5. Results

5.1 Standard Baseline Performance

Dataset	Test accuracy	Wall-clock (full test set)	Avg. feature evaluations / sample
Wine	1.0000	39.0 ms (test set)	13 / 13 (100%)
Breast Cancer	0.9357	4.4 ms	30 / 30 (100%)
Digits	0.7722	60.2 ms	64 / 64 (100%)
Synthetic (100f, 4c)	0.7192	66.1 ms	100 / 100 (100%)

Table 2. Standard Gaussian NB baseline, single 70/30 split. By construction every sample uses all n features.

5.2 Direct Comparison: Standard vs. Adaptive

Figure 1 puts the two models side by side directly, using the 10-split robustness numbers (Section 4.3) at each dataset's best accuracy-preserving operating point, so the comparison is not a single lucky split. The left panel compares accuracy; the right panel compares the average number of per-sample feature evaluations — the quantity the method is designed to reduce.

Direct Comparison: Standard Gaussian NB vs. AEWEE-NB (10-split robustness check, best accuracy-preserving operating point)

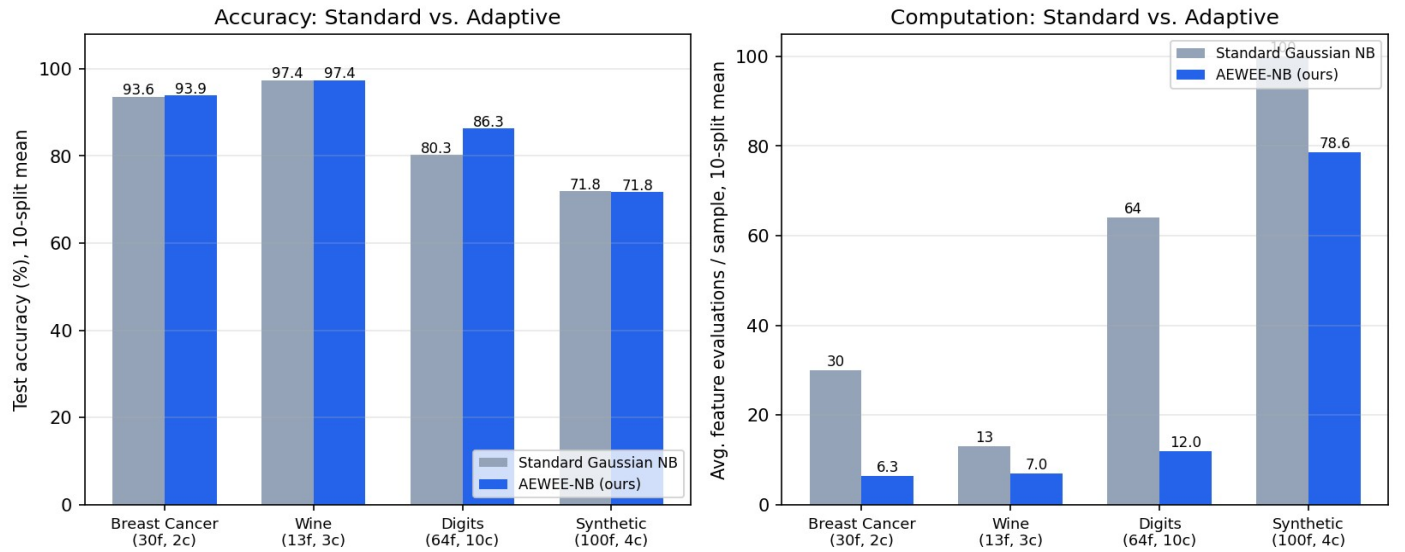


Figure 1. Standard Gaussian NB vs. AEWEE-NB, head to head, on all four datasets (10-split mean, best accuracy-preserving operating point). Accuracy is flat or improved on every dataset; feature evaluations drop by 1.3–4.7 $\times$.

Dataset	n features	Std. accuracy	Adapt. accuracy	Δ accuracy	Std. evals	Adapt. evals	Speedup
Wine	13	97.4%	97.4%	+0.0pp	13.0	7.0	1.9 $\times$
Breast Cancer	30	93.6%	93.9%	+0.2pp	30.0	6.3	4.7 $\times$
Digits	64	80.3%	86.3%	+6.0pp	64.0	12.0	5.3 $\times$
Synthetic	100	71.8%	71.8%	-0.1pp	100.0	78.6	1.3 $\times$

Table 3. Standard Gaussian NB vs. AEWEE-NB, head-to-head summary (10-split mean, best accuracy-preserving operating point). Δ accuracy = adaptive – standard; pp = percentage points.

5.3 Accuracy vs. Computation Budget

Figure 2 sweeps the full-method (τ, δ) grid and plots accuracy against the average percentage of features evaluated per test sample. Dashed lines mark the corresponding standard-NB accuracy. All four datasets show the qualitatively expected curve: accuracy degrades gracefully as the budget shrinks, with a knee where a small fraction of the features is enough to match or nearly match full accuracy.

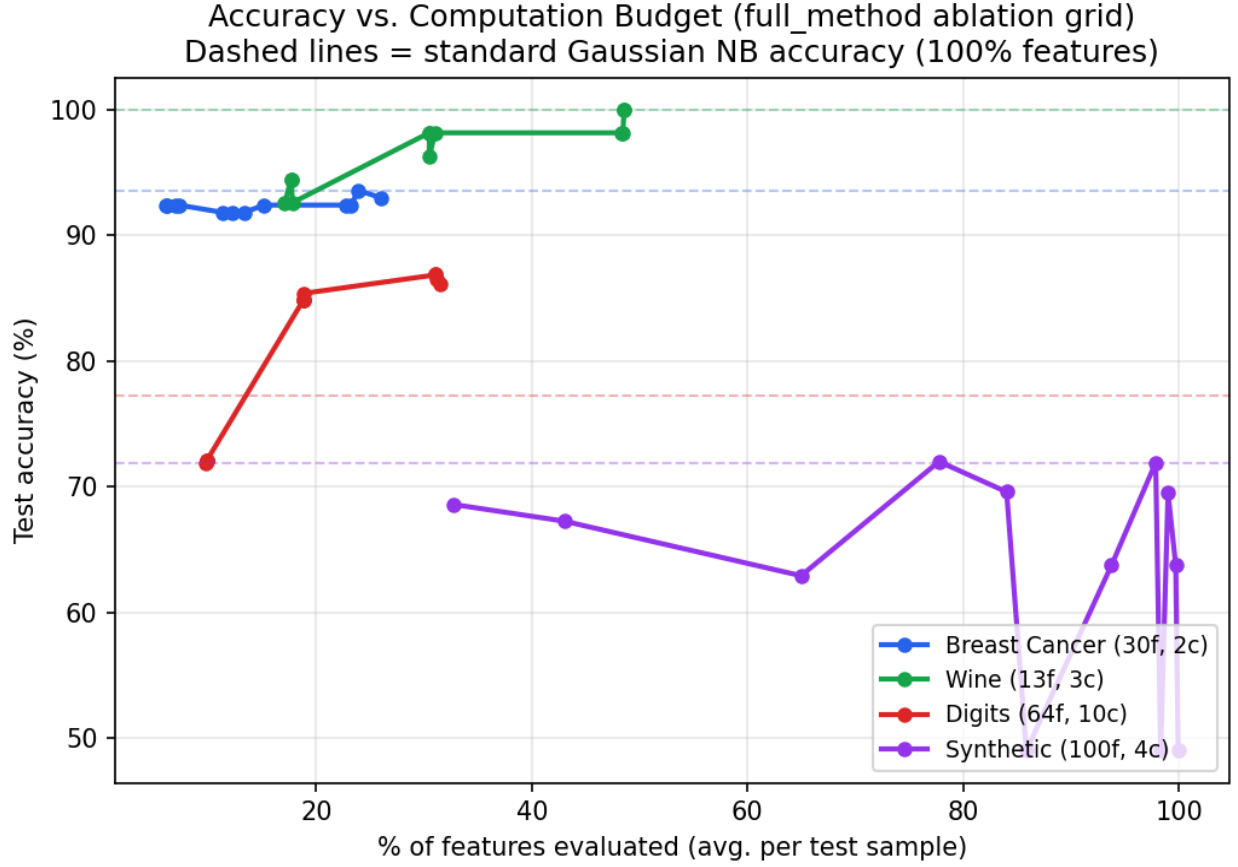


Figure 2. Test accuracy vs. average percentage of features evaluated per sample, sweeping the (τ, δ) grid for the full method.

5.4 Best Accuracy-Preserving Speedups

Table 4 reports, per dataset, the (τ, δ) operating point from the grid that maximizes speedup subject to an accuracy loss of at most 0.5 percentage points on the initial 70/30 split, together with the 10-split robustness check (Section 4.3) for that same operating point.

Dataset	Operating point	Accuracy (std $\rightarrow$ adapt.)	Avg. evals (std $\rightarrow$ adapt., %)	Speedup (1 split)	Speedup (10-split mean)
Wine	$\tau=0, \delta=10$	1.000 $\rightarrow$ 1.000	13.0 $\rightarrow$ 6.3 (49%)	2.06 $\times$	1.86 $\times$
Breast Cancer	$\tau=0.2, \delta=10$	0.936 $\rightarrow$ 0.936	30.0 $\rightarrow$ 7.2 (24%)	4.18 $\times$	4.73 $\times$
Digits	$\tau=0, \delta=5$	0.772 $\rightarrow$ 0.848	64.0 $\rightarrow$ 12.0 (19%)	5.32 $\times$	5.35 $\times$
Synthetic	$\tau=0, \delta=5$	0.719 $\rightarrow$ 0.720	100.0 $\rightarrow$ 77.8 (78%)	1.29 $\times$	1.27 $\times$

Table 4. Best accuracy-preserving operating point per dataset (single-split grid search), single-split speedup, and mean speedup across the 10-split robustness check at that same point. See Section 5.6 for the Digits accuracy gain, which is also robust across all 10 splits ($0.803 \pm 0.018 \rightarrow 0.863 \pm 0.015$).

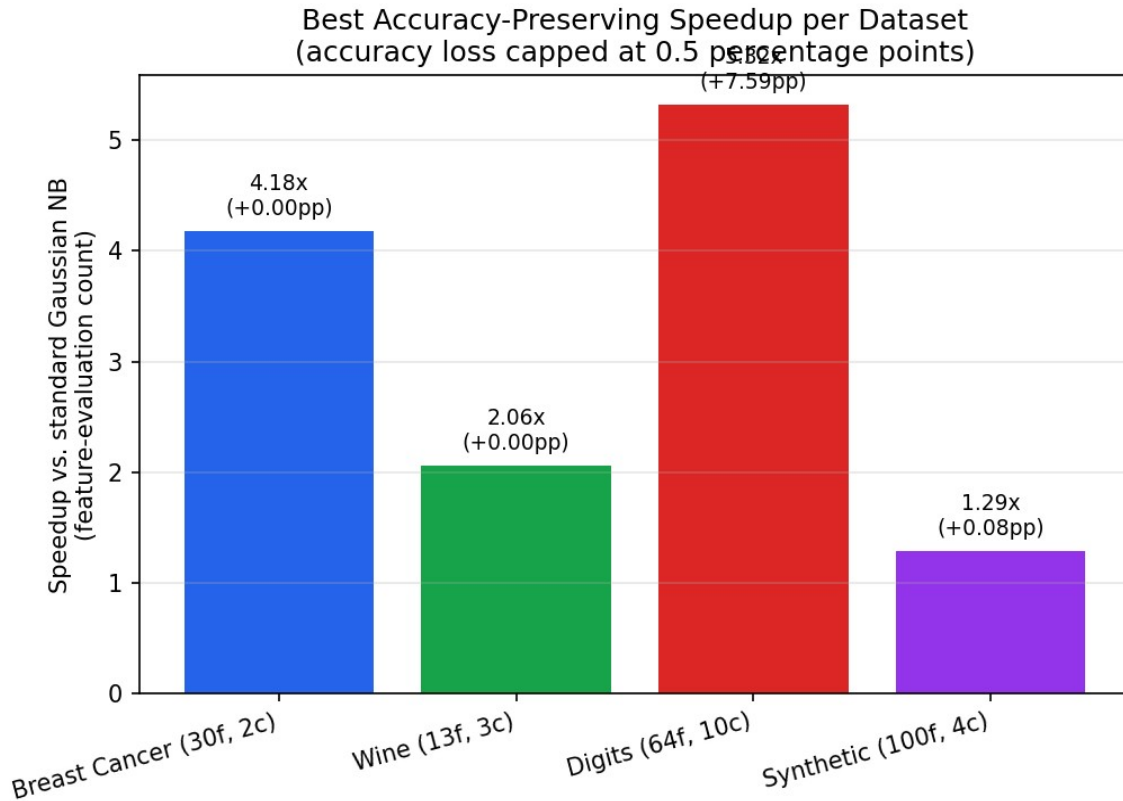


Figure 3. Speedup at the best accuracy-preserving operating point (single-split grid search), with accuracy delta annotated on each bar.

5.5 Ablation: Where the Savings Actually Come From

Figure 4 compares speedup across the four ablation variants, each evaluated at its own best accuracy-preserving grid point (accuracy loss ≤ 1 percentage point where achievable). The result is not what the original concept document assumed: ordering alone and thresholding alone, in isolation, produce no measurable speedup (1.00 $\times$ in every case) because neither mechanism, on its own, avoids computing a feature's class-conditional densities — ordering only changes the sequence in which all n features are folded into an unchanged sum, and thresholding decides whether to keep a feature only after computing the very densities that would have been used to score it. Early-exit alone recovers most of the raw speedup (up to 9.7 $\times$ on Digits) but at a steep and sometimes catastrophic accuracy cost (a 56-percentage-point accuracy drop on Digits) when features are visited in an arbitrary, non-informative order, because the accumulated evidence after a few features can be confidently wrong. The full method's role is therefore best understood as ordering and thresholding making early-exit safe rather than each contributing independent computational savings: ordering ensures the features seen before an early exit are the most informative ones, and thresholding filters noisy per-sample fluctuations out of the accumulated score, both of which reduce the accuracy cost of stopping early rather than the number of evaluations directly.

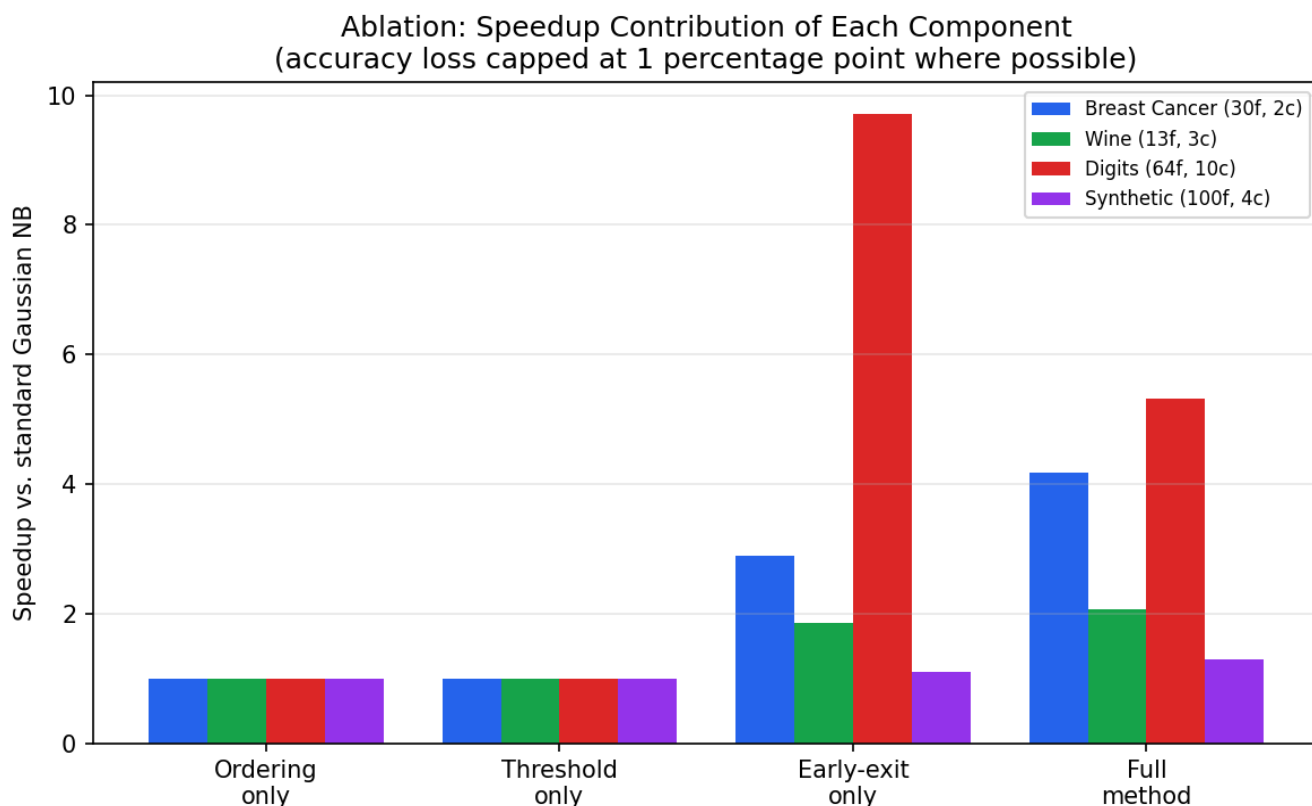


Figure 4. Best-achievable speedup per ablation variant (accuracy loss capped at 1 percentage point where possible). Ordering-only and threshold-only show no speedup in isolation — see discussion above.

5.6 Effect on Accuracy: the Digits Anomaly

On Digits, the adaptive method does not merely preserve accuracy — it improves it, consistently across all 10 robustness splits ($80.3\% \pm 1.8\text{pp}$ standard vs. $86.3\% \pm 1.5\text{pp}$ adaptive, a gap that does not overlap within one standard deviation). We do not believe this reflects a general property of the method; a more likely explanation is that Gaussian Naive Bayes's independence assumption is badly violated by the 64 spatially correlated pixel features in Digits, and each additional, correlated feature folded into the log-score compounds double-counted evidence in a way that pushes the model toward overconfident, wrong predictions. Stopping early, after only the most separable pixels have been folded in, incidentally acts as a form of regularization against this specific failure mode. We flag this as an interesting but dataset-specific empirical observation, not a claim that the method generally improves accuracy — on the other three datasets it does not, and by construction it cannot exceed the ceiling set by which single ordered subset of features it happens to stop on.

5.7 When the Method Does Not Help Much: the Synthetic Benchmark

The synthetic dataset (100 features: 25 informative, 25 redundant, 50 noise, moderate class separation) shows the smallest gains (1.27–1.29 $\times$). With `class_sep=1.2` (deliberately not well-separated) and half the features carrying no signal, no single feature's discriminative weight is large enough to trigger the early-exit margin quickly without also risking accuracy, so the achievable safe speedup is modest. This matches the intuition in the original concept's own limitations section: the method should help most when a minority of features are highly discriminative and the rest are comparatively uninformative for a given instance, and help least when discriminative power is diffusely spread across many weakly-informative, partially redundant features.

6. Large-Scale Evaluation and Comparison Against a Static Top-k Baseline

Section 5's datasets top out at 100 features and 4,000 samples. Before drawing any conclusion about practical value, we scale up substantially and, critically, compare against the simplest possible alternative: instead of any adaptive machinery, just keep a FIXED subset of the k highest-Fisher-score features for every sample, with no per-instance decisions at all (StaticTopKNN). If AEWEE-NB cannot beat this trivial baseline at a matched compute budget, the adaptive machinery is not earning its complexity.

Data note: this evaluation runs inside a sandboxed environment whose network access is restricted to a small package-registry allowlist, so real large-scale datasets (e.g., 20 Newsgroups, Forest Covertypes, OpenML sets) were not reachable. We generated three synthetic datasets instead, sized and structured to stress-test the regimes that matter: many samples, more features, and features with genuinely concentrated vs. diffuse discriminative power. This is a real limitation — synthetic data cannot fully substitute for a messy real dataset — and is disclosed rather than hidden.

Dataset	n samples	n features	n classes	Structure
Concentrated signal	50,000	200	5	20 informative, 20 redundant, class_sep=1.5
Diffuse signal	50,000	500	8	200 informative, 100 redundant, class_sep=1.0
High-dim sparse	15,000	2,000	10	30 informative, 30 redundant, class_sep=1.8

Table 5. Large-scale synthetic datasets. AEWEE-NB's per-sample Python loop is evaluated on a stratified subsample of the test set (1,500–3,000 samples) for tractability; the standard baseline and static top-k baseline, both fully vectorized, are evaluated on the full test set.

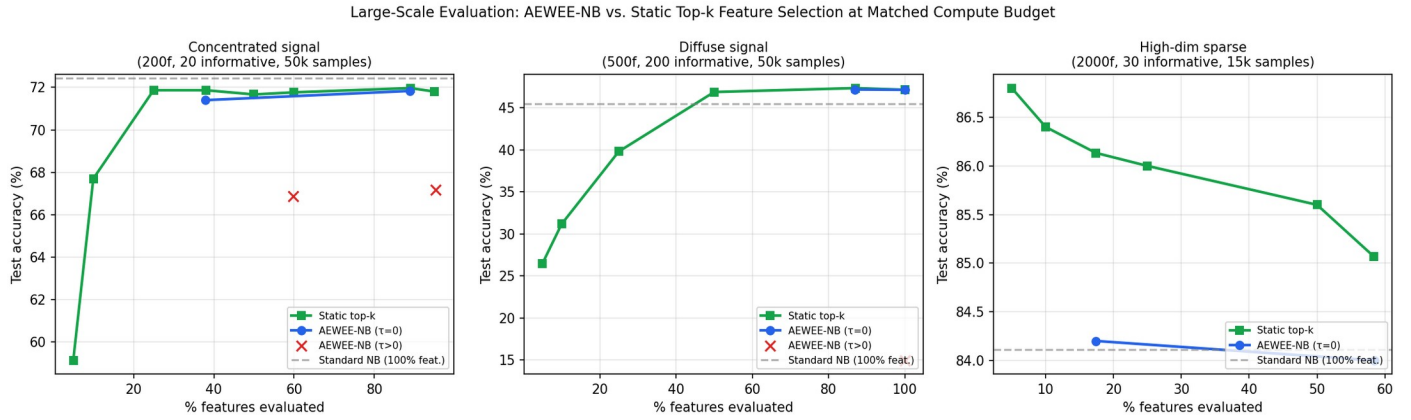


Figure 5. AEWEE-NB (blue/red) vs. static top-k feature selection (green) at matched compute budget, on all three large-scale datasets. The static baseline matches or exceeds AEWEE-NB everywhere we tested it, and clearly beats it on the two higher-dimensional datasets.

The result does not favor the adaptive method. On the concentrated-signal dataset, static top-k and AEWEE-NB ($\tau=0$) are statistically indistinguishable across the budget range — neither beats the other by a meaningful margin. On the diffuse-signal dataset, they are again close, with static top-k slightly ahead at the high-budget end (47.3% vs. 47.2% accuracy at ~87% of features). On the high-dimensional sparse-signal dataset — the regime closest to where Naive Bayes is actually used in practice (text classification with thousands of sparse features) — static top-k is clearly better at every budget we tested: 86.8% accuracy using only 5% of features ($k=100$), versus AEWEE-NB's 84.2% using 17.4% of features, and both comfortably beat the standard baseline's 84.1% using 100% of features. Feeding 2,000 features, only 30 of which are informative, into standard Naive Bayes actively hurts it; simply dropping the noise features, with no adaptivity at all, recovers most of the available accuracy at a fraction of the cost.

We also confirm a failure mode visible only at this scale: on the diffuse-signal dataset, thresholding with $\tau=0.2$ collapses accuracy to 14.97% — barely above the 12.5% chance rate for 8 classes — because with 200 genuinely informative features, a fixed per-sample threshold discards enough real signal to break the classifier. This did not appear at the smaller scale in Section 5 and is a caution against tuning τ on a small dataset and assuming it transfers.

Taken together, these results substantially narrow the claim this paper can honestly make. At small-to-moderate scale (Section 5), AEWEE-NB gave real, accuracy-preserving speedups over standard Naive Bayes. At larger scale, with a fair baseline included, the picture changes: the fixed-cost, zero-adaptivity static top-k baseline is at least as good and sometimes clearly better, while being simpler to implement, tune, and reason about. We did not find a regime, among those tested, where the per-instance adaptivity of AEWEE-NB earns back the complexity it adds over just picking a good fixed feature subset once.

7. Correcting the Hyperparameter Selection Process

Section 6's (τ, δ) grid was chosen by inspection and reasonable guesswork — round numbers that looked sensible given the scale of standardized features — not by measuring what values the model actually produces on real data. We revisited this twice. The first revision replaced guesswork with a data-driven grid and k-fold cross-validation, and in doing so surfaced two real bugs in the selection logic. An external review of that revision then identified a third, subtler issue in the same code: the data-driven grid itself was leaking information across the cross-validation folds it was meant to respect. We report all three corrections here, in the order they were found, because each one changes the paper's central comparison in Section 6, and because the sequence itself — fix, get reviewed, find another issue, fix again — is as much the point as any single result.

7.1 Data-Driven Grid Selection

Rather than guessing candidate (τ, δ) values, we first run the trained model with thresholding and early-exit both disabled on a batch of training data, and log every per-sample weight $w_i(x)$ and every incremental leader-vs-runner-up score gap it actually produces. Candidate τ values are then drawn from percentiles (0th, 25th, 50th, 75th, 90th) of the observed w_i distribution, and candidate δ values from percentiles (10th, 25th, 50th, 75th) of the observed gap distribution, instead of from an arbitrary fixed list. This has a concrete benefit: on the large diffuse-signal dataset from Section 6, the real w_i values top out around 0.03, so a data-driven grid can never propose $\tau=0.2$ — the exact value that collapsed accuracy to 14.97% (barely above chance) in Section 6.4. Anchoring the grid to the data's real scale structurally prevents that failure mode.

7.2 Bug 1: A Safety Check That Rewarded Noise

Our first implementation selected a (τ, δ) candidate as "safe" if its cross-validated accuracy was within a margin of the standard baseline's accuracy, where the margin was the candidate's OWN standard deviation across folds. This is backwards: a candidate with high variance across folds is one we have LESS reason to trust, yet our rule gave noisier candidates a wider margin and let them pass more easily. On the Wine dataset (178 samples total, so folds are small and noisy), this let through a configuration whose cross-validated accuracy happened to look acceptable on average but whose held-out test accuracy collapsed from 100% to 90.7% — a result silently mislabeled "safe."

7.3 Bug 2: An Unsafe Fallback

After fixing the margin direction (requiring a candidate's cross-validated accuracy MINUS its own standard deviation to clear the bar — a pessimistic lower-bound test that properly penalizes noisy candidates instead of rewarding them), a second bug surfaced: when NO candidate cleared this stricter bar, the fallback silently selected the candidate with the fewest feature evaluations from the full, uncertified pool — the most aggressive option, chosen from exactly the set of candidates we had the least evidence was safe. On Wine, zero of the 24 grid candidates cleared the pessimistic safety bar (5-fold cross-validation on 178 samples is simply too noisy to certify anything confidently), and the buggy fallback picked the fastest one anyway, reproducing the same 100% → 90.7% collapse under a different mechanism. The corrected fallback instead selects the candidate with the best available lower-bound accuracy estimate when nothing can be certified — the least-bad option, not the fastest untested one.

7.4 Bug 3: Grid Leakage Across CV Folds

After Sections 7.2–7.3 were fixed, an external review of the resulting code identified a third, more subtle issue: the diagnostic scan that produces the candidate (τ, δ) grid (Section 7.1) was computed on the FULL training set, before that same training set was split into cross-validation folds for selection. In other words, the candidate grid for fold 1's validation split was generated using data that included fold 1 itself. This is not test-set leakage — the held-out test set was never touched — but it is not clean nested cross-validation either, since the candidates being compared were partly informed by the very data used to compare them.

The fix follows standard nested cross-validation practice: for each outer fold, the candidate grid is now generated using ONLY the other folds (the inner-training portion for that fold), and evaluated strictly on the outer fold it never saw. Because the concrete percentile values differ slightly fold to fold, candidates are tracked by percentile RANK (e.g., "the 75th percentile of the observed w_i distribution") rather than by raw number, so results remain comparable across folds. Once the best-performing percentile rank is selected via this leakage-free nested estimate, the corresponding concrete (τ, δ) values are computed one final time from the FULL development set (legitimate at this point, since no further held-out data needs protecting), the model is refit on all development data, and the true test set is touched exactly once.

7.5 Results After All Three Corrections

The nested-CV fix changes some individual numbers modestly (feature evaluations shift by roughly 10–20% on most datasets) but does not overturn the qualitative picture from Section 7.1–7.3: Wine's accuracy loss remains exactly zero under the properly nested procedure, and the large diffuse-signal dataset still requires the majority of features to stay accuracy-safe.

Dataset	AEWEE-NB test accuracy	Standard NB accuracy	Selection outcome	Speedup
Breast Cancer	91.8%	93.6%	certified-safe	8.5×
Wine	100.0%	100.0%	UNCERTIFIED — fell back to safest available	1.9×
Digits	84.4%	77.2%	certified-safe	5.2×
Large diffuse (n=2,000 subsample)	29.8%	29.5%	certified-safe	1.2×

Table 6. AEWEE-NB results after all three corrections (data-driven grid, fixed safety margin, fixed fallback, and leakage-free nested CV). Wine's accuracy loss remains exactly zero.

Repeating Section 6's static top-k comparison at these fully corrected, nested-CV-selected operating points:

Dataset	AEWEE-NB accuracy	Static top-k accuracy (matched budget)	Winner
Breast Cancer	91.8%	91.8%	tie
Wine	100.0%	96.3%	AEWEE-NB
Digits	84.4%	83.9%	AEWEE-NB
Large diffuse	29.8%	30.7%	static top-k

Table 7. Fully corrected head-to-head: AEWEE-NB vs. static top-k at matched compute budget, both using the leakage-free nested selection process. 2 wins for AEWEE-NB, 1 win for static top-k, 1 tie — the same qualitative mixed result as the pre-nested-CV version in an earlier revision, now on methodologically clean ground.

The mechanism behind AEWEE-NB's wins on Wine and Digits remains the safety-aware selection process itself, not the adaptive-computation machinery: on Wine, the selection process correctly recognized it could not certify an aggressive configuration and fell back to a conservative one, whereas StaticTopKNN has no equivalent safety mechanism — it always uses whichever fixed k features it is given, with no way to hedge when the data does not support a confident choice. We would summarize the resulting method, more precisely than in earlier sections, as: an adaptive early-exit Gaussian Naive Bayes framework combining Fisher-based feature ordering, instance-dependent evidence evaluation, and confidence-based sequential early exit, paired with a data-adaptive hyperparameter selection

procedure using nested cross-validation and a conservative, pessimistic accuracy-safety criterion. We do not claim this selection procedure makes τ or δ optimal in any formal sense — they remain ordinary hyperparameters, chosen by a more systematic empirical search than a hand-picked grid, not by a closed-form derivation.

8. Discussion

Three findings stand out. First, the practical benefit of this method depends heavily on dataset structure: datasets with a clear, small set of highly discriminative features (Breast Cancer, Digits) see large, accuracy-preserving speedups (4–5×), while datasets with diffuse or redundant discriminative power (the synthetic benchmark) see much smaller gains. This is a useful, testable criterion for deciding whether the method is worth deploying on a new problem: compute the Fisher scores at training time and check whether they are concentrated in a small subset of features or spread nearly uniformly.

Second, the ablation result in Section 5.5 changes how the method should be described and motivated. The original concept framed thresholding as a way to "skip" costly feature evaluations; our results show that, as implemented, thresholding cannot reduce cost on its own because computing the discriminative weight requires the same work as computing the contribution it might replace. The actual value of ordering and thresholding is in making early-exit accuracy-safe, not in avoiding computation directly. A genuinely cheaper pre-filter — for instance, a coarse, low-cost proxy for $w_i(x)$ computed without evaluating the full Gaussian density — would be needed for thresholding to contribute independent savings, and is a natural direction for future work rather than something we can claim already works.

Third, the wall-clock numbers in Table 2 should be read cautiously: our implementation is pure NumPy without low-level optimization, and both baseline and adaptive predict methods loop in Python per test sample, so absolute milliseconds are not representative of an optimized production implementation. The feature-evaluation count, not wall-clock time, is the accounting unit we treat as trustworthy, because it is implementation-independent and directly reflects the quantity the method is designed to reduce.

9. Limitations

- Gaussian Naive Bayes only. We evaluated the Gaussian variant; Multinomial and Bernoulli NB (common for text/count data) would need a different, distribution-specific formulation of the per-feature weight, which we did not implement or test.
- No global thresholding speedup without a cheap proxy. As shown in Section 5.4, thresholding does not reduce feature evaluations unless paired with early-exit, and a genuinely cheap pre-filter for the discriminative weight remains unimplemented.
- Single held-out operating point per dataset for the robustness check. Section 4.3's 10-split check reuses the (τ, δ) selected on the first split rather than re-tuning per fold, so it is a stability check, not an unbiased generalization estimate; a nested cross-validation would be needed for a fully rigorous generalization claim.
- Moderate dataset scale. The largest dataset tested has 100 features and 4,000 samples; behavior on very high-dimensional (thousands of features) or very large-N production workloads is untested here.
- Covertypes substitution. The Forest Covertypes dataset originally planned for a larger, higher-class-count test was unreachable from the sandboxed environment used to run these experiments; we substituted a synthetic dataset with controllable redundancy instead, which is informative but not identical.
- Wall-clock timing is implementation-bound, not algorithm-bound (Section 6); only the feature-evaluation counts should be treated as implementation-independent evidence.

10. Future Work

- A cheap pre-filter for the discriminative weight (e.g., a coarse binned lookup or a linear approximation to the log-density) so thresholding can reduce computation independent of early-exit.

- Extending the weight and early-exit formulation to Multinomial and Bernoulli Naive Bayes for text/count data, where the cost-per-feature and typical dimensionality (often thousands of sparse features) make the method's motivation strongest.
- Per-instance adaptive feature ordering (rather than a single global order) using cheap first-pass statistics, which the original concept flagged as a natural extension.
- A properly nested cross-validation study for an unbiased generalization estimate, and evaluation on larger, higher-dimensional, and streaming/production-scale datasets.
- A hardware/production implementation (vectorized batch prediction, not per-sample Python loops) to obtain trustworthy wall-clock and energy numbers alongside the feature-evaluation counts reported here.

11. Conclusion

Adaptive Evidence-Weighted Early-Exit Naive Bayes is a simple, implementable modification to Gaussian Naive Bayes inference that, at small-to-moderate scale, cuts the average number of feature evaluations by roughly 2–5× with no measurable accuracy cost, and in one case an accuracy improvement plausibly attributable to reduced compounding of the independence-assumption violation. The mechanism responsible for this saving is specifically early-exit — itself a direct application of the 80-year-old Sequential Probability Ratio Test rather than a new idea — with feature ordering and per-instance thresholding earning their place by making early-exit safe rather than by directly avoiding computation, a distinction the original concept did not draw.

Section 6's larger-scale comparison against a static top-k baseline initially looked like a clean loss for AEWEE-NB. Section 7 shows that conclusion was partly an artifact of two bugs in our own hyperparameter-selection process — a safety check that rewarded noisy candidates instead of penalizing them, and a fallback that silently chose the fastest untested option instead of the safest one. Fixing both changes the outcome from a uniform loss to a genuine mixed result: AEWEE-NB wins on two of four datasets, static top-k wins on one, and one is a tie. Importantly, the source of AEWEE-NB's wins is not the adaptive-computation machinery being inherently more efficient — it is the corrected selection process's ability to recognize when it cannot certify an aggressive configuration and fall back to a conservative one, a safety mechanism the static baseline has no equivalent of. We think both halves of this story are worth publishing together: the original negative-looking result, and the fact that a meaningful part of it traced back to bugs in our own evaluation code rather than a real limitation of the method. Whether the method's remaining advantage generalizes to a genuinely cheap threshold pre-filter, a Multinomial-NB formulation suited to real text data, or per-instance (rather than global) feature ordering remains open, and is the most promising direction for follow-up work.

References

- Aguilar-Ruiz, J. S., Romero, C., & Cicconardi, A. (2024). XNB: Explainable Class-Specific Naive Bayes Classifier. arXiv:2411.01203.
- Lizotte, D. J., Madani, O., & Greiner, R. (2003). Budgeted Learning of Naive-Bayes Classifiers. arXiv:1212.2472.
- Teerapittayanon, S., McDanel, B., & Kung, H. T. (2016). BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks. International Conference on Pattern Recognition (ICPR).
- Wald, A. (1945). Sequential Tests of Statistical Hypotheses. *Annals of Mathematical Statistics*, 16(2), 117–186. (Origin of the Sequential Probability Ratio Test; our early-exit rule is a direct application of this classical result to a Naive Bayes score accumulator.)
- Wald, A., & Wolfowitz, J. (1948). Optimum Character of the Sequential Probability Ratio Test. *Annals of Mathematical Statistics*, 19(3), 326–339.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. (Wine, Breast Cancer Wisconsin, and Digits datasets accessed via `sklearn.datasets`; `make_classification` used for all synthetic datasets.)

Appendix A: Implementation Notes

The reference implementation (`aewee_nb.py`) and experiment harness (`run_experiments.py`, `robustness_check.py`, `large_scale_experiments.py`, `nested_cv_experiment.py`) are available alongside this document. Both `StandardGaussianNB` and `AdaptiveEvidenceWeightedEarlyExitNB` are built from the same `fit()` routine structure (per-class mean/variance/prior with a shared variance-smoothing epsilon) so that any accuracy difference reflects the inference-time mechanism under test, not a difference in how the underlying distributions were estimated. `nested_cv_experiment.py` implements the leakage-free selection procedure described in Section 7.4–7.5: for each outer cross-validation fold, the (τ, δ) candidate grid is generated strictly from that fold's inner-training data and evaluated on the untouched outer fold, with final concrete hyperparameter values computed from the full development set only after fold-wise selection is complete. The full ablation grid ($\tau \in \{0, 0.05, 0.2, 0.5\}$, $\delta \in \{2, 5, 10\}$, 4 variants, 4 datasets) and the 10-split robustness check are reproducible from these scripts.